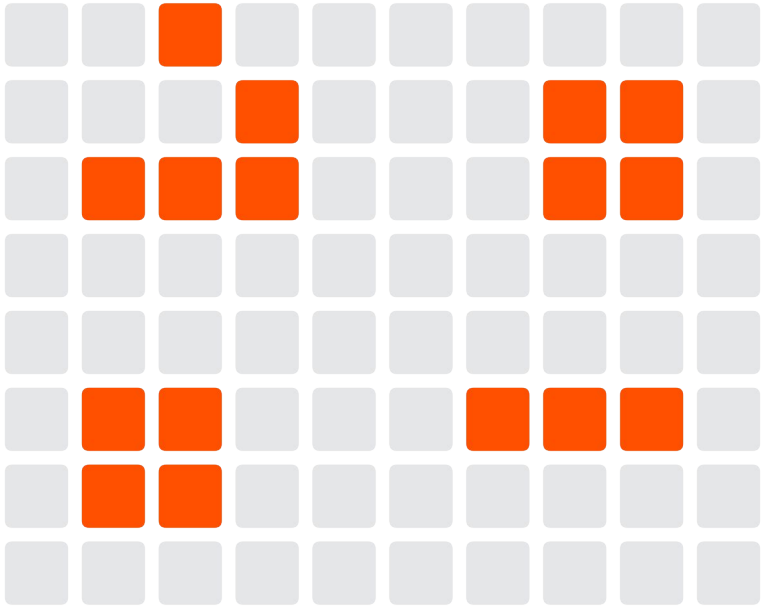
JavaScript: Arrays and Functions

Software 2 Autumn 2026 - Week 2

Building the Game of Life as we go



This week

Week 2

Lectures

1. Project kick-off, paper prototypes
2. JS: Interactive programs, control structures
3. **JS: Arrays, functions** (today)

Submissions

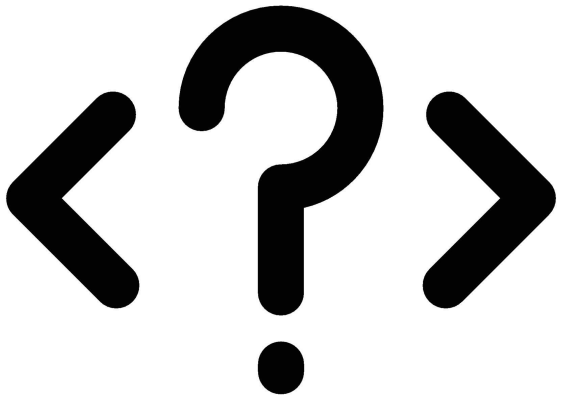
- **1.11** HTML/CSS exercise
- **1.11** JS module exercises, modules 1 and 2

Today

Plan for the lecture

1. Game of Life: what we will build
2. JavaScript arrays
3. JavaScript functions

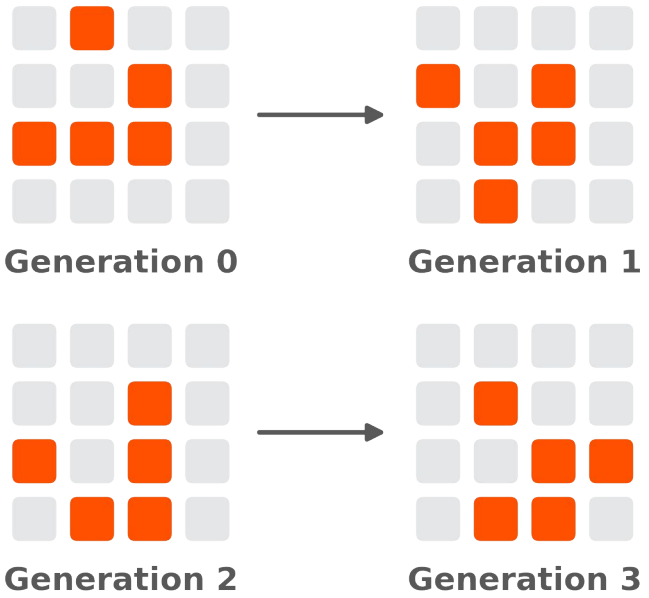
We build the game step by step, one small piece at a time



Game of Life

What is it?

- Invented by John Conway in 1970
- A grid of cells, each **alive** or **dead**
- No players: the start pattern decides everything
- All cells update at once, step by step



Game of Life

The rules

- Look at the 8 neighbours of a cell
- **Alive** cell with 2 or 3 alive neighbours stays alive, otherwise it dies
- **Dead** cell with exactly 3 alive neighbours comes alive





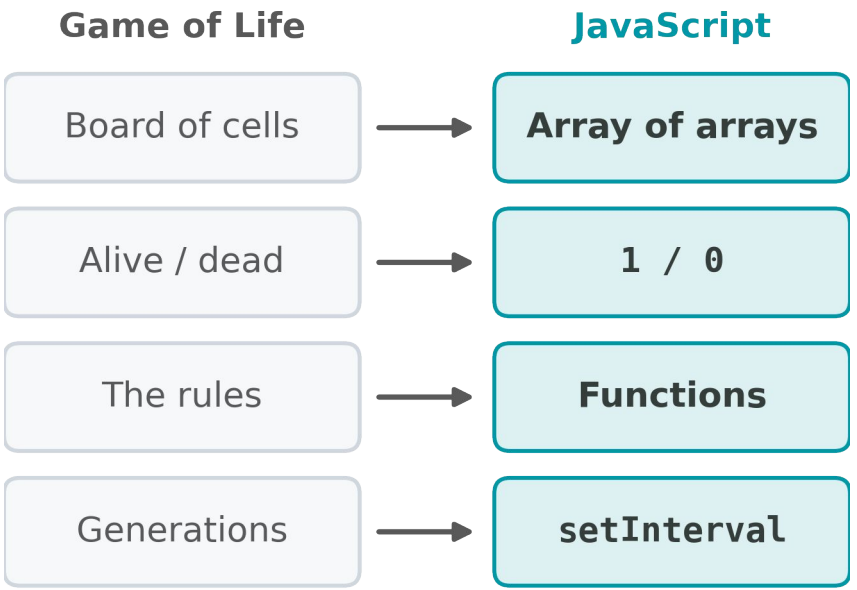




Game of Life

What we will build

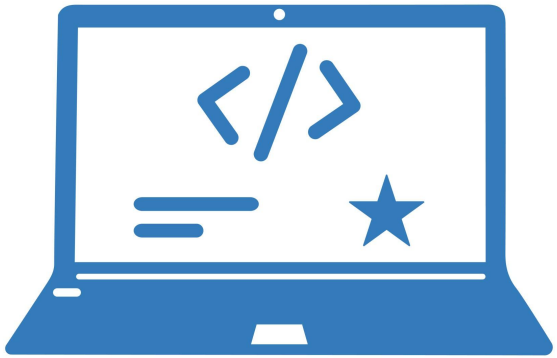
- A grid of cells shown in the browser
- A new generation every 0.2 seconds
- Start from HTML/CSS base code
- We write the JavaScript step by step



Arrays

Overview

- Creating arrays
- Adding items to arrays
- Iterating over arrays
- Arrays of arrays: the Game of Life grid

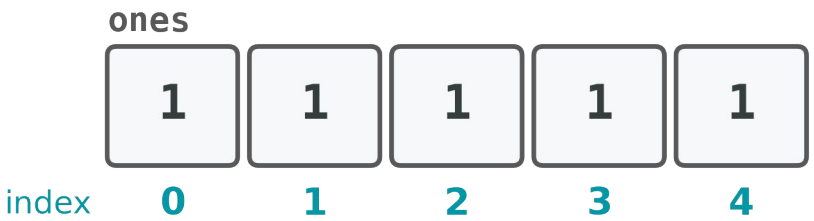


Arrays

What is an array?

- Like a Python list: values in order
- Every value has an index (its position)
- The first index is 0
- `ones.length` is the number of items

```
let zeroes = []
zeroes[0] = 0
zeroes[1] = 0
let ones = [1, 1, 1, 1, 1]
```



Arrays

Looping over an array

- As in Python, a for loop visits every item
- Counter loop: use the index `i`
- `for...of` loop: gives each item directly

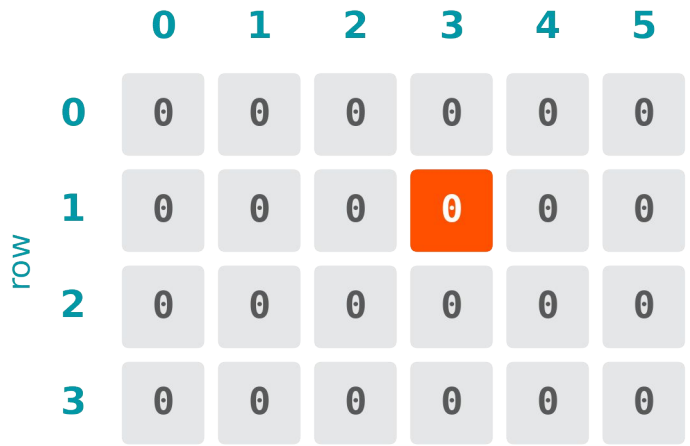
```
for (let i = 0; i < ones.length; i++) {
    console.log(ones[i])
}

for (let one of ones) {
    console.log(one)
}
```

Arrays

Game of Life · Step 1: the grid

- Mimic the board with an array of arrays
- Each element of `grid` is one row
- `grid[row][column]` reads one cell
- Build it with a loop inside a loop



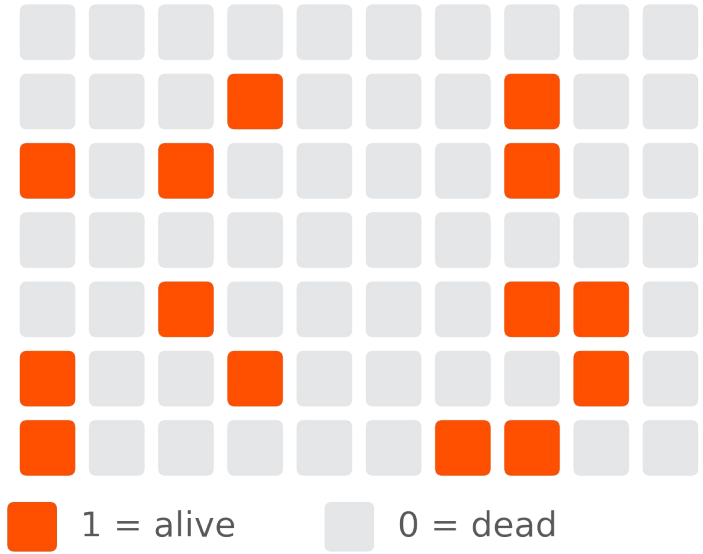
`grid[1][3]`

row 1, column 3

Arrays

Game of Life · Step 2: random start

- Start with a mix of alive (1) and dead (0) cells
- For each cell, generate a random number between 0 and 1
- Over 0.7: the cell is 1, otherwise 0



Functions

Defining a function

- Reusable piece of code, as in Python
- Define once, call as often as you like
- `return` sends a value back

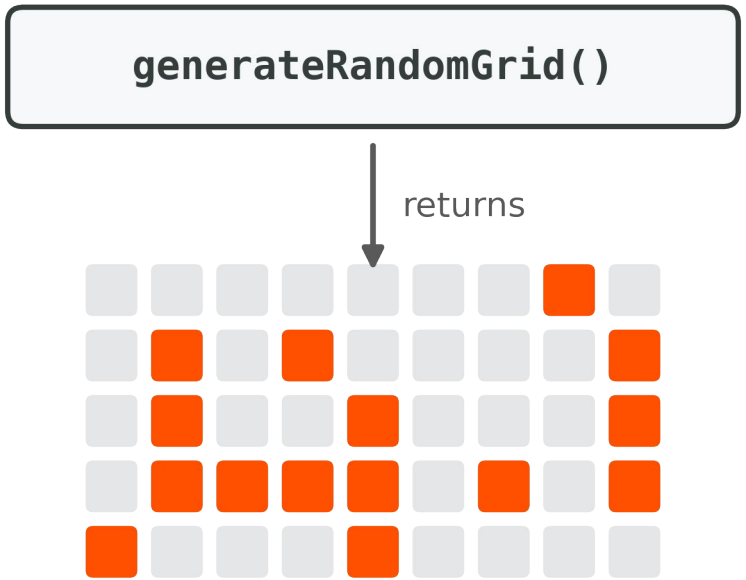
```
function greet() {
    console.log("Well, hello!");
    return;
}

greet()  // call it
```

Functions

Game of Life · Step 3: grid as a function

- Put the grid creation inside a function
- `generateRandomGrid()` builds the grid and returns it
- Call it whenever we need a fresh random grid



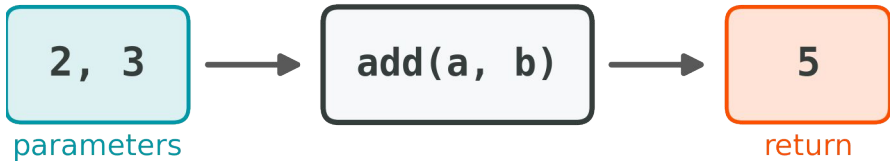
Functions

Parameters and return values

- Parameters: values the function receives
- `return`: the value it gives back
- Store the result in a variable

```
function add(a, b) {
    return a + b
}

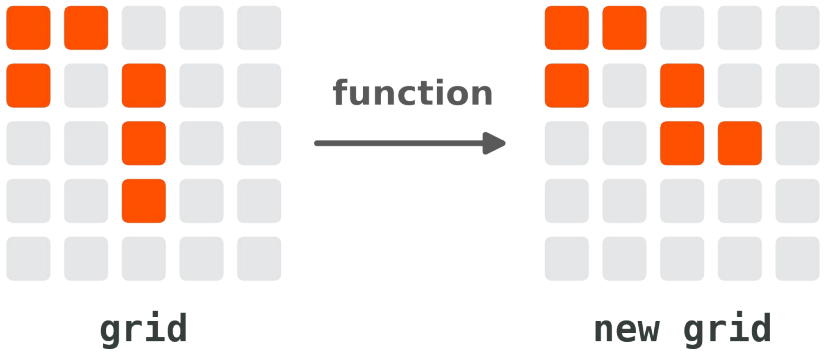
let sum = add(2, 3)   // 5
```



Functions

Game of Life · Step 4: the next generation

- Every step we generate a new grid, made by the rules
- A function takes the previous grid and makes the next one
- Step 1: Make a new function, that takes the grid as a parameter, loops over the rows and columns, and print the values

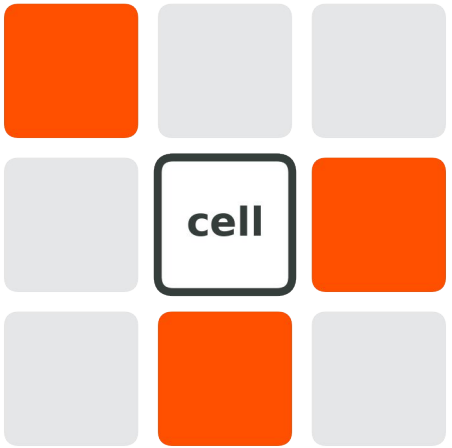


previous state in, next state out

Functions

Game of Life · Step 5: alive neighbours

- For each cell we need its number of alive neighbours
- `countAliveNeighbors(r, c)` returns it
- The result is a number from 0 to 8

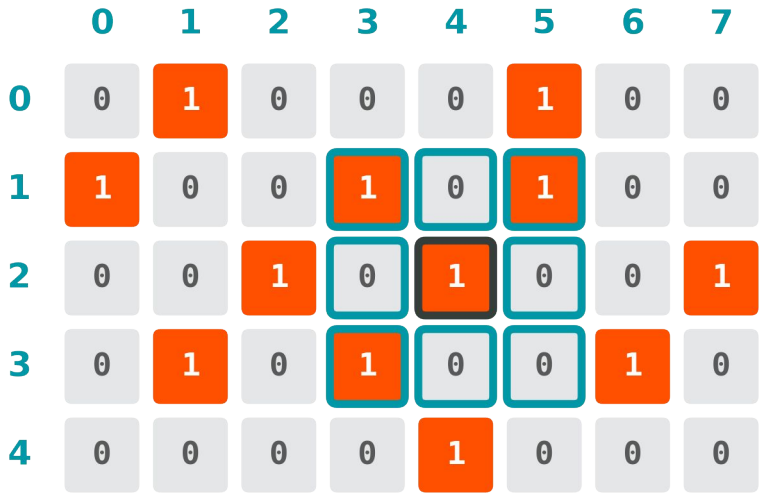


3 of the 8 neighbours are alive

Functions

Game of Life · Step 5: an example

- Cell at row 2, index 4
- Row 1: index 3, 4, 5
- Row 2: index 3 and 5
- Row 3: index 3, 4, 5
- Count how many are alive (value 1)



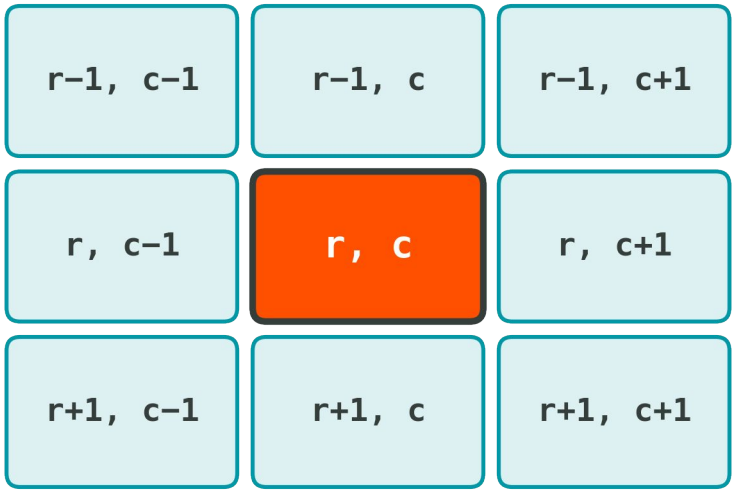
alive neighbours of (2, 4): 3

dark = the cell, teal = its neighbours

Functions

Game of Life · Step 5: in general

- Cell at row `r`, index `c`
- Row `r-1`: indexes `c-1`, `c`, `c+1`
- Row `r`: indexes `c-1`, `c+1`
- Row `r+1`: indexes `c-1`, `c`, `c+1`

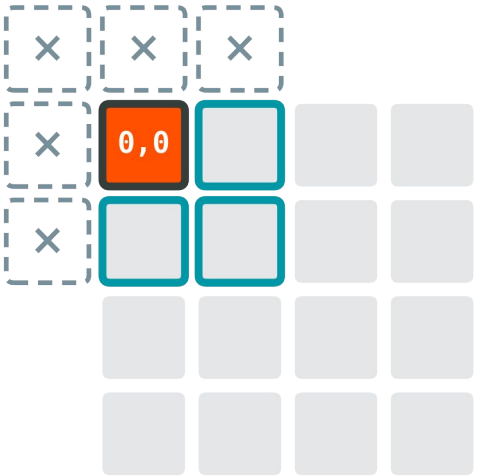


Functions

Game of Life · Step 5: your turn

- Write `countAliveNeighbors(r, c)`
- Count the surrounding cells that are alive (1)
- Watch the edges: they have fewer neighbours

Think through how this could be done



Corner cell: only 3 neighbours

inside the grid; dashed = outside

Functions

Game of Life · Step 6: next state of a cell

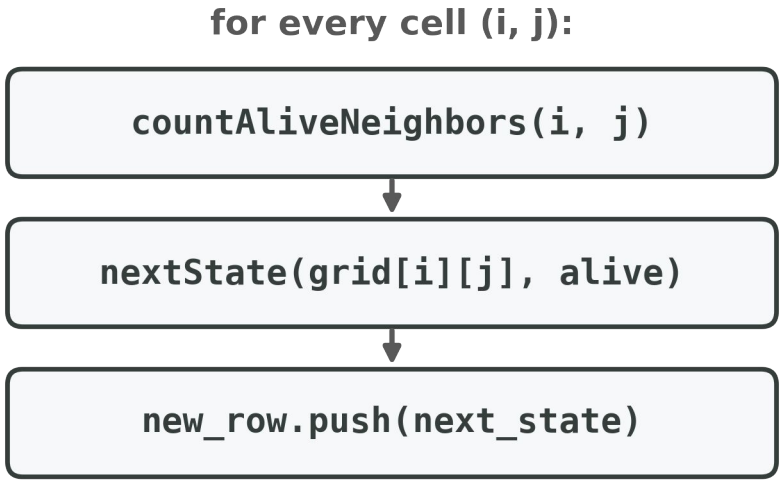
- Write `nextState(cell, neighbors)`
- It gets the cell state (0 or 1) and its alive neighbours
- It returns the state in the next generation

Cell now	Alive neighbours	Next state
alive (1)	0 or 1	**dead (0)**
alive (1)	2 or 3	**alive (1)**
alive (1)	4 or more	**dead (0)**
dead (0)	exactly 3	**alive (1)**
dead (0)	anything else	**dead (0)**

Functions

Game of Life · Step 7: the new grid

- Create an empty new grid
- For every cell: count alive neighbours, then get the next state
- Save each result in the new grid, not the old one




```
after the loops:  grid = new_grid
```

Functions

Game of Life · Step 8: timer

- `step()` makes the next grid and renders it
- `setInterval` runs `step` every 200 ms
- Result: a running Game of Life!

```
function step() {
    calculateNextGrid()
    render(grid)
}

generateRandomGrid()
timerId = setInterval(step, 200)
```

Recap

What we learned

- Arrays: ordered values, index from 0
- Array of arrays: a grid
- Functions: parameters in, value out
- Game of Life: grid + rules + timer

```
let a = [0, 1, 1]
a[0]          // first item
a.length      // 3
for (let x of a) { }
grid[1][2]    // row 1, column 2
function f(p) { return p }
```